МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ

Федеральное государственное автономное
образовательное учреждение высшего образования
“Национальный исследовательский университет ИТМО”

**ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ**

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине
“Распределённые системы хранения данных”

Вариант: 74.

Работу выполнил:
Студент группы P3311
Болорболд Аригуун
Преподаватель:
Николаев Владимир Владиславович

г. Санкт-Петербург
2025 г.

Содержимое

1 Текст задания **3**
1.1 Цель работы 3
1.2 Требования к выполнению работы 3
1.3 Этап 1. Конфигурация 3
1.4 Этап 2. Симуляция и обработка сбоя 3
1.5 Этап 3. Восстановление 3

2 Выполнение **4**
2.1 Этап 1. Конфигурация 5
2.2 Этап 2. Симуляция и обработка сбоя 9
2.2.1 Подготовка 9
2.2.2 Сбой 11
2.2.3 Обработка 11
2.3 Этап 3. Восстановление 13

3 Вывод **15**

1 Текст задания

1.1 Цель работы

Цель работы - ознакомиться с методами и средствами построения отказоустойчивых решений на базе СУБД Postgres; получить практические навыки восстановления работы системы после отказа.
Работа рассчитана на двух человек и выполняется в три этапа: настройка, симуляция и обработка сбоя, восстановление.

1.2 Требования к выполнению работы

- В качестве хостов использовать одинаковые виртуальные машины.
- В первую очередь необходимо обеспечить сетевую связность между ВМ.
- Для подключения к СУБД (например, через psql), использовать отдельную виртуальную или физическую машину.
- Демонстрировать наполнение базы и доступ на запись на примере не менее, чем двух таблиц, столбцов, строк, транзакций и клиентских сессий.

1.3 Этап 1. Конфигурация

Развернуть postgres на двух узлах в режиме **потоковой репликации. Не использовать дополнительные пакеты**. Продемонстрировать доступ в режиме чтение/запись на основном сервере. Продемонстрировать, что новые данные синхронизируются на резервный сервер.

1.4 Этап 2. Симуляция и обработка сбоя

2.1 Подготовка:

- Установить несколько клиентских подключений к СУБД.
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

2.2 Сбой:
Симулировать неожиданное отключение основного узла — **выполнить Power Off виртуальной машины**.

2.3 Обработка:

- Найти и продемонстрировать в логах релевантные сообщения об ошибках.
- Выполнить переключение (failover) на резервный сервер.
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

1.5 Этап 3. Восстановление

- Восстановить работу основного узла — откатить действие, выполненное с виртуальной машиной на этапе 2.2.
- Актуализировать состояние базы на основном узле - накатить все изменения данных, выполненные на этапе 2.3.
- Восстановить исправную работу узлов в исходной конфигурации (в соответствии с этапом 1).
- Продемонстрировать состояние данных и работу клиентов в режиме чтение/запись.

2 Выполнение

Подготовка окружения
Структура:

```
├── docker-compose.yml
├── primary
│   ├── conf
│   │   ├── pg_hba.conf
│   │   └── postgresql.conf
│   ├── Dockerfile
│   ├── init
│   │   └── init-primary.sh
│   └── scripts
│       ├── init-db.sql
│       ├── read_client.sh
│       └── write_client.sh
└── standby
    ├── conf
    │   ├── pg_hba.conf
    │   └── postgresql.conf
    ├── Dockerfile
    ├── init
    │   └── init-standby.sh
    └── scripts
        ├── auto_promote.sh
        └── read_client.sh
```

Хосты сконфигурированы через `docker-compose.yml`:

```yaml
services:
  primary:
    container_name: primary
    build:
      context: ./primary
    restart: unless-stopped
    ports:
      - "5432:5432"
    environment:
      - PGDATA=/var/lib/postgresql/data
      - PGENCODING=UTF8
      - PGLOCALE=en_US.UTF8
      - PGUSERNAME=postgres
      - POSTGRES_PASSWORD=postgres
    volumes:
      - ./primary/data:/var/lib/postgresql/data
    networks:
      - pg_net

  standby:
    container_name: standby
    build:
      context: ./standby
    restart: unless-stopped
    ports:
      - "5433:5432"
    depends_on:
      - primary
    environment:
      - PGDATA=/var/lib/postgresql/data
      - PGENCODING=UTF8
      - PGLOCALE=en_US.UTF8
      - PGUSERNAME=postgres
      - POSTGRES_PASSWORD=postgres
    volumes:
      - ./standby/data:/var/lib/postgresql/data
    networks:
      - pg_net

networks:
  pg_net:
    driver: bridge
```

Соединение между хостами настроено через сеть `pg_net` и порты 5432 и 5433 соответственно.

2.1 Этап 1. Конфигурация

Ведущий узел
`Dockerfile`

```
FROM postgres:latest
COPY conf/* /etc/postgresql/
COPY scripts/* /home/scripts/
COPY init/init-primary.sh /home/init/init-primary.sh
RUN chmod +x /home/scripts/read_client.sh
RUN chmod +x /home/scripts/write_client.sh
RUN chmod +x /home/init/init-primary.sh
```

init-primary.sh

```
#!/bin/bash
set -e

psql -v ON_ERROR_STOP=1 --username "postgres" -c
"CREATE ROLE replicator WITH REPLICATION PASSWORD 'replicator_password' LOGIN;"
psql -v ON_ERROR_STOP=1 --username "postgres" -f "/home/scripts/init-db.sql"

cp /etc/postgresql/postgresql.conf "$PGDATA/postgresql.conf"
cp /etc/postgresql/pg_hba.conf "$PGDATA/pg_hba.conf"

echo "Configuration files copied!"
```

postgresql.conf

```
listen_addresses = '*'
wal_level = hot_standby
max_wal_senders = 10
archive_mode = on
archive_command = 'echo "dummy command, archive_command called"'
hot_standby = on
```

pg_hba.conf

```
# TYPE  DATABASE        USER            ADDRESS                 METHOD
local   all             all                                     trust
host    all             all             0.0.0.0/0               md5
host    replication     replicator      0.0.0.0/0               md5
```

Ведомый узел
Dockerfile

```
FROM postgres:latest
COPY conf/* /etc/postgresql/
COPY scripts/* /home/scripts/
COPY init/init-standby.sh /docker-entrypoint-initdb.d/init-standby.sh
RUN chmod +x /home/scripts/read_client.sh
RUN chmod +x /docker-entrypoint-initdb.d/init-standby.sh
RUN chmod +x /home/scripts/auto_promote.sh
```

init-standby.sh

```
#!/bin/bash
set -e

until pg_isready -h primary -p 5432 -U postgres; do
  echo "Waiting for primary to be ready..."
  sleep 2
done

pg_ctl stop -D "$PGDATA"

rm -rf "$PGDATA"/*
echo "Data directory cleaned up"

PGPASSWORD='replicator_password' pg_basebackup -h primary
-D /var/lib/postgresql/data -U replicator -v -P --wal-method=stream
echo "Base backup completed"

touch "$PGDATA/standby.signal"
chown -R postgres:postgres "$PGDATA"

cp /etc/postgresql/postgresql.conf "$PGDATA/postgresql.conf"
cp /etc/postgresql/pg_hba.conf "$PGDATA/pg_hba.conf"
echo "Conf files copied"

pg_ctl -D "$PGDATA" start
```

postgresql.conf

```
listen_addresses = '*'
wal_level = hot_standby
hot_standby = on
primary_conninfo = 'host=primary port=5432 user=replicator
password=replicator_password'
max_wal_senders = 10
archive_mode = on
archive_command = 'echo "dummy command, archive_command called"'
```

pg_hba.conf

```
# TYPE  DATABASE        USER            ADDRESS                 METHOD
local   all             all                                     trust
host    all             all             0.0.0.0/0               md5
host    replication     replicator      0.0.0.0/0               md5
```

Запуск:
Ведущего узла:

```
docker-compose up -d --build primary
```

```
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker-compose up -d --build primary
[+] Building 1.1s (14/14) FINISHED
 => [internal] load local bake definitions                                                                    0.0s
 => => reading from stdin 602B                                                                                0.0s
 => [internal] load build definition from Dockerfile                                                          0.0s
 => => transferring dockerfile: 297B                                                                          0.0s
 => [internal] load metadata for docker.io/library/postgres:latest                                            0.1s
 => [internal] load .dockerignore                                                                             0.0s
 => => transferring context: 2B                                                                               0.0s
 => [1/7] FROM docker.io/library/postgres:latest@sha256:29574e213ffaf433ca9c9441a5919868b85ec04e7a32377f48edd3d3f9721  0.0s
 => => resolve docker.io/library/postgres:latest@sha256:29574e213ffaf433ca9c9441a5919868b85ec04e7a32377f48edd3d3f9721  0.0s
 => [internal] load build context                                                                             0.0s
 => => transferring context: 317B                                                                             0.0s
 => CACHED [2/7] COPY conf/* /etc/postgresql/                                                                 0.0s
 => CACHED [3/7] COPY scripts/* /home/scripts/                                                                0.0s
 => CACHED [4/7] COPY init/init-primary.sh /home/init/init-primary.sh                                         0.0s
 => CACHED [5/7] RUN chmod +x /home/scripts/read_client.sh                                                    0.0s
 => CACHED [6/7] RUN chmod +x /home/scripts/write_client.sh                                                   0.0s
 => CACHED [7/7] RUN chmod +x /home/init/init-primary.sh                                                      0.0s
 => exporting to image                                                                                        0.2s
 => => exporting layers                                                                                       0.0s
 => => exporting manifest sha256:fe23147dfb61bdcbe2c5a3bda2919640e6b3dc4aabedeb2cb8604861bc3efb08             0.0s
 => => exporting config sha256:e9aa900d60c4d1a06d9662fcf870df1448c1cc58fa316d28c51c289676953aac               0.0s
 => => exporting attestation manifest sha256:ed7cfd41635b6f2de562e8ee5bc98f4feac7f86e24a34ff810dd68e07f01431b  0.0s
 => => exporting manifest list sha256:9423295b1c6d9c309dc928bd2f371d8fcab297ca9541b85937218b9280ac6389        0.0s
 => => naming to docker.io/library/lab4-primary:latest                                                        0.0s
 => => unpacking to docker.io/library/lab4-primary:latest                                                     0.0s
 => resolving provenance for metadata file                                                                    0.0s
[+] Running 3/3
 ✔lab4-primary         Built                                                                                  0.0s
 ✔Network lab4_pg_net  Created                                                                                0.1s
 ✔Container primary    Started                                                                                0.5s
```

Ожидаем лога "database system is ready to accept connections":

```
Success. You can now start the database server using:

    pg_ctl -D /var/lib/postgresql/data -l logfile start
```

```
initdb: warning: enabling "trust" authentication for local connections
initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb.
waiting for server to start....2025-09-29 10:37:39.527 UTC [49] LOG:  starting PostgreSQL 17.6 (Debian 17.6-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled
by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
2025-09-29 10:37:39.530 UTC [49] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2025-09-29 10:37:39.558 UTC [52] LOG:  database system was shut down at 2025-09-29 10:37:37 UTC
2025-09-29 10:37:39.607 UTC [49] LOG:  database system is ready to accept connections
```

Инициализация ведущего узла (`init_primary.sh`):

```
docker exec -it primary bash -c "/home/init/init-primary.sh"
docker restart primary
```

```
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker exec -it primary bash -c "/home/init/init-primary.sh"
CREATE ROLE
CREATE DATABASE
You are now connected to database "test" as user "postgres".
CREATE TABLE
CREATE TABLE
INSERT 0 2
INSERT 0 2
Configuration files copied!
```

Ведомого узла:

```
docker-compose up -d --build standby
```

```
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker-compose up -d --build standby
[+] Building 1.4s (25/25) FINISHED
 => [internal] load local bake definitions                                                                    0.0s
 => => reading from stdin 1.12kB                                                                              0.0s
 => [primary] resolving provenance for metadata file                                                          0.0s
[+] Running 4/4
 ✔lab4-primary       Built                                                                                    0.0s
 ✔lab4-standby       Built                                                                                    0.0s
 ✔Container primary  Started                                                                                  1.4s
 ✔Container standby  Started                                                                                  0.8s
```

Наполнение базы

По требованию был создан `init-db.sql`:

```
CREATE DATABASE test;
\c test;

CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(id),
    product VARCHAR(255)
);

INSERT INTO users (name) VALUES ('Alice'), ('Bob');
INSERT INTO orders (user_id, product) VALUES (1, 'Laptop'), (2, 'Smartphone');
```

Проверим данные на ведущем узле:

```
docker exec -it primary psql -U postgres -d test -c "SELECT * FROM users;"
docker exec -it primary psql -U postgres -d test -c
"INSERT INTO users (name) VALUES ('Bradley');"
```

```
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker exec -it primary psql -U postgres -d test -c "SELECT * FROM users;"
 id | name
----+-------
  1 | Alice
  2 | Bob
(2 rows)

PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker exec -it primary psql -U postgres -d test -c "INSERT INTO users (name) VALUES ('Bradley');"
INSERT 0 1
```

Проверим режим чтения на ведомом узле:

```
docker exec -it standby psql -U postgres -d test -c "SELECT * FROM users;"
```

```
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker exec -it standby psql -U postgres -d test -c "SELECT * FROM users;"
 id |  name
----+---------
  1 | Alice
  2 | Bob
  3 | Bradley
(3 rows)
```

Проверим режим записи на ведомом узле (должен выдать ошибку):

```
docker exec -it standby psql -U postgres -d test -c
"INSERT INTO users (name) VALUES ('Bradley');"
```

```
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker exec -it standby psql -U postgres -d test -c "INSERT INTO users (name) VALUES ('Bradley');"
ERROR:  cannot execute INSERT in a read-only transaction
```

2.2 Этап 2. Симуляция и обработка сбоя

2.2.1 Подготовка

Скрипты для симуляции чтения и записи из ведущего и чтение из ведомого узла: `read_client.sh`

```
#!/bin/bash
while true; do
    psql -U postgres -d test -c "SELECT * FROM users;"
    psql -U postgres -d test -c "SELECT * FROM orders;"
    sleep 2
done
```

write_client.sh

```
#!/bin/bash
while true; do
psql -U postgres -d test -c "INSERT INTO users (name) VALUES
('User_$(date +%s)');"
items=("TV" "Mouse" "Keyboard" "HDMI cable")
random_item=${items[$RANDOM % ${#items[@]}]}
last_user_id=$(psql -U postgres -d test -t -c "SELECT id FROM users
ORDER BY id DESC LIMIT 1;")
psql -U postgres -d test -c "INSERT INTO orders (user_id, product)
VALUES ($last_user_id, '$random_item');"
sleep 2
done
```

Запуск:

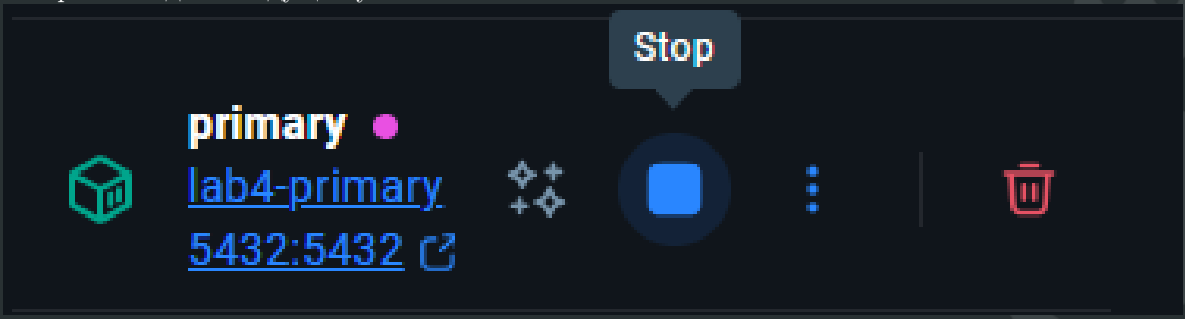
```
docker exec -it primary bash /home/scripts/read_client.sh
docker exec -it primary bash /home/scripts/write_client.sh
docker exec -it standby bash /home/scripts/read_client.sh
```

Ожидаем что на ведомом узле чтение будет автоматически показывать новые данные из ведущего узла:

```
PS C:\Users\USER> docker exec -it standby bash /home/scripts/read_client.sh
 id |      name
----+-----------------
  1 | Alice
  2 | Bob
  3 | Bradley
  4 | User_1759141986
(4 rows)

 id | user_id |  product
----+---------+------------
  1 |       1 | Laptop
  2 |       2 | Smartphone
  3 |       4 | TV
(3 rows)

 id |      name
----+-----------------
  1 | Alice
  2 | Bob
  3 | Bradley
  4 | User_1759141986
  5 | User_1759141988
(5 rows)

 id | user_id |  product
----+---------+------------
  1 |       1 | Laptop
  2 |       2 | Smartphone
  3 |       4 | TV
  4 |       5 | Mouse
(4 rows)
```

2.2.2 Сбой

Симулируем ошибку диска на основном узле — просто выключаем виртуальную машину, на котором находится ведущий узел.



При этом чтение и запись в ведущий узел будет невозможно:

```
 id |      name
----+-----------------
  1 | Alice
  2 | Bob
  3 | Bradley
  4 | User_1759135947
  5 | User_1759135949
  6 | User_1759135952
  7 | User_1759135954
  8 | User_1759135956
  9 | User_1759135958
 10 | User_1759135960
 11 | User_1759135962
 12 | User_1759135965
(12 rows)

 id | user_id |  product
----+---------+------------
  1 |       1 | Laptop
  2 |       2 | Smartphone
  3 |       4 | Mouse
  4 |       5 | Keyboard
  5 |       6 | HDMI cable
  6 |       7 | Keyboard
  7 |       8 | Mouse
  8 |       9 | HDMI cable
  9 |      10 | Mouse
 10 |      11 | TV
 11 |      12 | Mouse
(11 rows)

psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL:  the database system is shutting down
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL:  the database system is shutting down
```

```
PS C:\Users\USER> docker exec -it primary bash /home/scripts/write_client.sh
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL:  the database system is shutting down
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL:  the database system is shutting down
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL:  the database system is shutting down
```

2.2.3 Обработка

В ведомом узле видим логи:

```
2025-09-29 10:37:51.131 GMT [72] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2025-09-29 10:37:51.131 GMT [72] LOG:  listening on IPv6 address "::", port 5432
2025-09-29 10:37:51.138 GMT [72] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2025-09-29 10:37:51.176 GMT [75] LOG:  database system was interrupted; last known up at 2025-09-29 10:37:41 GMT
..2025-09-29 10:37:54.065 GMT [75] LOG:  starting backup recovery with redo LSN 0/9000028, checkpoint LSN 0/9000080, on timeline ID 2
2025-09-29 10:37:54.069 GMT [75] LOG:  entering standby mode
2025-09-29 10:37:54.117 GMT [75] LOG:  redo starts at 0/9000028
2025-09-29 10:37:54.123 GMT [75] LOG:  completed backup recovery with redo LSN 0/9000028 and end LSN 0/9000120
2025-09-29 10:37:54.124 GMT [75] LOG:  consistent recovery state reached at 0/9000120
2025-09-29 10:37:54.125 GMT [72] LOG:  database system is ready to accept read-only connections
2025-09-29 10:37:54.125 GMT [75] LOG:  invalid record length at 0/A000060: expected at least 24, got 0
 done
server started

2025-09-29 10:37:54.149 GMT [76] LOG:  started streaming WAL from primary at 0/A000000 on timeline 2
waiting for server to shut down...2025-09-29 10:37:54.154 GMT [72] LOG:  received fast shutdown request
.2025-09-29 10:37:54.161 GMT [72] LOG:  aborting any active transactions
```

Но при этом чтение работает:

```
docker exec -it standby bash /home/scripts/read_client.sh
```

```
 id |      name
----+-----------------
  1 | Alice
  2 | Bob
  3 | Bradley
  4 | User_1759141986
  5 | User_1759141988
  6 | User_1759141990
  7 | User_1759141992
  8 | User_1759141994
  9 | User_1759141996
 10 | User_1759141999
 36 | Charlie
(11 rows)

 id | user_id |  product
----+---------+------------
  1 |       1 | Laptop
  2 |       2 | Smartphone
  3 |       4 | TV
  4 |       5 | Mouse
  5 |       6 | HDMI cable
  6 |       7 | HDMI cable
  7 |       8 | Keyboard
  8 |       9 | Keyboard
  9 |      10 | TV
(9 rows)

 id |      name
----+-----------------
  1 | Alice
  2 | Bob
  3 | Bradley
  4 | User_1759141986
  5 | User_1759141988
  6 | User_1759141990
  7 | User_1759141992
  8 | User_1759141994
  9 | User_1759141996
 10 | User_1759141999
 36 | Charlie
(11 rows)

 id | user_id |  product
----+---------+------------
  1 |       1 | Laptop
  2 |       2 | Smartphone
  3 |       4 | TV
  4 |       5 | Mouse
  5 |       6 | HDMI cable
  6 |       7 | HDMI cable
  7 |       8 | Keyboard
  8 |       9 | Keyboard
  9 |      10 | TV
(9 rows)
```

При работающем ведомом узле запустим скрипт автоматического `promote`: `auto-promote.sh`:

```bash
#!/bin/bash
set -e

PRIMARY_HOST="primary"
PRIMARY_PORT=5432
PG_CONNECT_TIMEOUT=3
CHECK_INTERVAL=5

while true; do
    if ! pg_isready -h $PRIMARY_HOST -p $PRIMARY_PORT -t $PG_CONNECT_TIMEOUT -q;
    then
        echo "Primary is unavailable. Attempting promote..."
        if [ -f $PGDATA/standby.signal ]; then
            if pg_ctl promote -D $PGDATA; then
                echo "Promote successful. This node is now master."
                exit 0
            else
                echo "Promote failed!" >&2
                exit 1
            fi
        else
            echo "Not in standby mode. Cannot promote." >&2
            exit 1
        fi
    fi
    sleep $CHECK_INTERVAL
done
```

Проверим чтение и запись на standby после `promote`:

```bash
docker exec -it standby psql -U postgres -d test -c
"INSERT INTO users (name) VALUES ('Charlie');"
docker exec -it standby psql -U postgres -d test -c
"SELECT * FROM users;"
```

```
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker exec -it standby psql -U postgres -d test -c "INSERT INTO users (name) VALUES ('Charlie');"
INSERT 0 1
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker exec -it standby psql -U postgres -d test -c "SELECT * FROM users;"
 id |      name
----+-----------------
  1 | Alice
  2 | Bob
  3 | Bradley
  4 | User_1759135947
  5 | User_1759135949
  6 | User_1759135952
  7 | User_1759135954
  8 | User_1759135956
  9 | User_1759135958
 10 | User_1759135960
 11 | User_1759135962
 12 | User_1759135965
 13 | User_1759135967
 36 | Charlie
(14 rows)
```

Как и ожидалось, запись и чтение работает.

2.3 Этап 3. Восстановление

Восстанавливаем работу primary. В консоли primary:

```
docker exec -it primary bash
su postgres
pg_basebackup -P -X stream -c fast -h standby -U replicator -D ~/backup
# password: **********
rm -rf /var/lib/postgresql/data/

mv ~/backup/* /var/lib/postgresql/data/
```

```
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker exec -it primary bash
root@59220e1023f7:/# su postgres
postgres@59220e1023f7:/$ pg_basebackup -P -X stream -c fast -h standby -U replicator -D ~/backup
Password:
30839/30839 kB (100%), 1/1 tablespace
postgres@59220e1023f7:/$ rm -rf /var/lib/postgresql/data
rm: cannot remove '/var/lib/postgresql/data': Device or resource busy
postgres@59220e1023f7:/$ mv ~/backup/* /var/lib/postgresql/data
postgres@59220e1023f7:/$
```

Выходим из контейнера ведущего узла.
В консоли ведомого:

```
docker exec -it standby bash
touch /var/lib/postgresql/data/standby.signal
```

```
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker exec -it standby bash
root@ba4f13c09d55:/# touch /var/lib/postgresql/data/standby.signal
root@ba4f13c09d55:/#
```

Выходим из контейнера standby.
Останавливаем standby, а потом перезапускаем ведущего и пересобираем образ ведомого узла:

```
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker-compose stop standby
[+] Stopping 1/1
 ✔Container standby  Stopped                                                                    0.6s
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker-compose up -d primary
[+] Running 1/1
 ✔Container primary  Running                                                                    0.0s
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker-compose up -d --build standby
```

Итоговая проверка:

```
PS C:\ITMO\Software Engineering\Distributed Database Systems\Lab4> docker exec -it primary bash /home/scripts/read_client.sh
 id |      name
----+-----------------
  1 | Alice
  2 | Bob
  3 | Bradley
  4 | User_1759141986
  5 | User_1759141988
  6 | User_1759141990
  7 | User_1759141992
  8 | User_1759141994
  9 | User_1759141996
 10 | User_1759141999
 36 | Charlie
(11 rows)

 id | user_id |  product
----+---------+------------
  1 |       1 | Laptop
  2 |       2 | Smartphone
  3 |       4 | TV
  4 |       5 | Mouse
  5 |       6 | HDMI cable
  6 |       7 | HDMI cable
  7 |       8 | Keyboard
  8 |       9 | Keyboard
  9 |      10 | TV
(9 rows)
```

```
PS C:\Users\USER> docker exec -it primary bash /home/scripts/write_client.sh
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
INSERT 0 1
```

```
^C
PS C:\Users\USER> docker exec -it standby bash /home/scripts/read_client.sh
 id |      name
----+-----------------
  1 | Alice
  2 | Bob
  3 | Bradley
  4 | User_1759141986
  5 | User_1759141988
  6 | User_1759141990
  7 | User_1759141992
  8 | User_1759141994
  9 | User_1759141996
 10 | User_1759141999
 36 | Charlie
 37 | User_1759142331
(12 rows)

 id | user_id |  product
----+---------+------------
  1 |       1 | Laptop
  2 |       2 | Smartphone
  3 |       4 | TV
  4 |       5 | Mouse
  5 |       6 | HDMI cable
  6 |       7 | HDMI cable
  7 |       8 | Keyboard
  8 |       9 | Keyboard
  9 |      10 | TV
 36 |      37 | TV
(10 rows)
```

3 Вывод

В ходе лабораторной работы я научился восстановлению базы данных (в физическом смысле) при отказоустойчивом работе системы БД PostgreSQL на 2 связанных узлах.